0269. Alien Dictionary

ard | Category: Advanced Graphs | LeetCode: https://leetcode.com/problems/alien

Problem

- Words are sorted in an unknown alphabet. Derive a valid letter order, or "" if the input is contradictory.

Intuition

- Compare each pair of adjacent words: the first differing letter gives an edge a → b (a comes before b).
- A topological sort of the letters is a valid alphabet; a cycle means contradiction.
- Special invalid case: a word followed by its own proper prefix ("abc" before "ab").

Brute Force

- Try all permutations of the letters: O(26!).

Optimal Approach

- Add every letter as a node.
- For adjacent words w1, w2: find the first differing index; add edge; if none and w1 is longer, return "".
- Kahn's topological sort; if not all letters are output, return "".

Worked Example

```
["wrt","wrf","er","ett","rftt"].

wrt / wrf  -> t < f
wrf / er   -> w < e
er  / ett  -> r < t
ett / rftt -> e < r
topological order: w e r t f -> "wertf"
```

Java Solution

```java
public class AlienDictionary {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> adj = new LinkedHashMap<>();
        Map<Character, Integer> indeg = new HashMap<>();
        for (String w : words)
            for (char c : w.toCharArray()) { adj.putIfAbsent(c, new LinkedHashSet<>()); indeg.putIfAbsent(c, 0); }
        for (int i = 0; i + 1 < words.length; i++) {
            String a = words[i], b = words[i + 1];
            int len = Math.min(a.length(), b.length()), j = 0;
            while (j < len && a.charAt(j) == b.charAt(j)) j++;
            if (j == len) {
                if (a.length() > b.length()) return "";
                continue;
            }
            if (adj.get(a.charAt(j)).add(b.charAt(j))) indeg.merge(b.charAt(j), 1, Integer::sum);
        }
        Deque<Character> q = new ArrayDeque<>();
        for (char c : adj.keySet()) if (indeg.get(c) == 0) q.add(c);
        StringBuilder sb = new StringBuilder();
        while (!q.isEmpty()) {
            char c = q.poll();
            sb.append(c);
            for (char nx : adj.get(c)) if (indeg.merge(nx, -1, Integer::sum) == 0) q.add(nx);
        }
        return sb.length() == adj.size() ? sb.toString() : "";
    }
}
```

Complexity

- Time: O(total characters). Space: O(1) for 26 letters (plus edges).

Pitfalls

- Only the first differing character gives information; stop comparing after it.
- Avoid duplicate edges inflating in-degree (use a set per node).